

The `lutorch_ex` library

lookup-table layers: tables, projections, cartridges, and results

(draft)

2026-10-05 — draft v2

In one paragraph. `lutorch_ex` is a small PyTorch library for *lookup-table layers*: a learnable map built from many discrete tables, each addressed by a cheap locality-sensitive hash of its input. It has four parts, and this report walks through them in order: the tables and their hash (§1); a projection wrapper that turns a stack of tables into a drop-in dense-width layer (§2); *cartridges* — interchangeable read mathematics behind one fixed contract, in three generations (§3–4), one of which is quantised to int8 and ships with a deployment path (§4.4–4.5); and the shared machinery of regularisers, precision and kernels (§5). §6 reports a seven-configuration sweep at 48k steps; §7 lists what is still open. Reference material is in Appendix A.

1 The idea: tables addressed by an LSH

A lookup table (LUT) replaces a multiply-accumulate with an *address* and a *read*. Hash the input to a small integer with a locality-sensitive hash (LSH) — here, a handful of sign bits — and fetch the stored vector at that address. The construction comes from Izhikevich’s *Spiking Manifesto* [1]: a spiking network stores a vast number of synapses but touches only those of the neurons that fired, which is exactly what a table read does, and the *order* of spikes rather than their magnitude carries the information, which is exactly what a sign-bit address encodes.

One table. Work in a space $z \in \mathbb{R}^{d_{\text{in}}}$ (Figure 1). An *anchor* is an index into z . A table owns `nap` probes; probe j is either an anchor pair (a_j, b_j) , giving $u_j = z[a_j] - z[b_j]$, or a single anchor a_j , giving $u_j = z[a_j]$. Each probe contributes one bit $[u_j > 0]$, so an address $c \in [0, K)$ has `nap` bits and the table has $K = 2^{\text{nap}}$ cells, each holding a learnable row $W[c] \in \mathbb{R}^{d_{\text{out}}}$. The magnitudes $|u_j|$ are the *margins*: how far each bit is from flipping. The smallest one, $u^* = \min_j |u_j|$, marks the least certain bit, and flipping that bit gives the *neighbour* cell c' — the one the input would land in under the smallest perturbation. Every cartridge in this library uses c , u , and c' and nothing else from the input.

Two kinds of probe. Which kind a layer uses is fixed when the layer is built. In the default, pair form, every probe compares two coordinates of z : this is the manifesto’s Eq. 1, where the bit records which of two neurons fired first. In the single form every probe tests the sign of one coordinate instead.

Many tables, many heads. A layer stacks `tph` tables per group over G groups, called heads. A head reads its own slice of the input and writes its own slice of the output, and its output is the

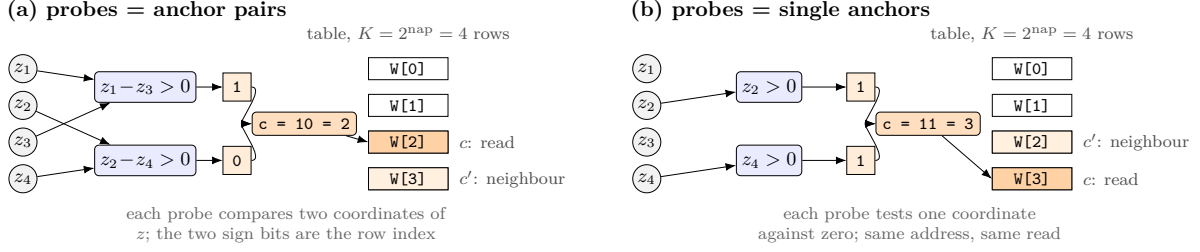


Figure 1: One lookup table in each of the two modes, $\text{nap} = 2$. A probe yields one sign bit; the bits, most significant first, form the row index c , which selects one learnable row of the $K = 2^{\text{nap}}$ -row table. The probe with the smallest margin is the least certain, and flipping its bit names the neighbour row c' (lighter), which the soft and surrogate reads also touch. (a) Anchor pairs compare two coordinates of z ; (b) single anchors test one coordinate against zero. A layer sums many such tables.

sum of its **tph** table reads (weighted, in some cartridges). A layer may also fan several heads into one shared output by summing them.

2 ProjectionLUT: input and output projections

On its own a table reads a narrow per-group vector of a space whose axes it cannot choose. **ProjectionMHL** (the “**ProjectionLUT**” wrapper, Figure 2) puts a plain linear map on each side of the cartridge and so turns it into a plain dense-width layer. The wrapper takes an input width and an output width that are independent and may differ; in every configuration used here both are the model width, $d_{\text{model}} \rightarrow d_{\text{model}}$. Writing h_{in} and h_{out} for the number of input and output heads and d_{in} , d_{out} for their widths ($h_{\text{in}} = h_{\text{out}} = G$ in the usual per-head case¹):

$$z = \text{compress}(x) \in [B, h_{\text{in}}, d_{\text{in}}], \quad y = \text{cartridge}(z) \in [B, h_{\text{out}}, d_{\text{out}}], \quad \text{out} = \text{decompress}(y).$$

- **The input projection shapes the hashing.** A probe is also a hyperplane test on z — $z[a] - z[b] = 0$ or $z[a] = 0$, a plane through the origin with coefficients ± 1 — and with **compress** in front, the same probe on the projected vector is an *arbitrary* learned hyperplane $w^\top x + b$ on the original x . This is how fixed probes get general decision boundaries (the manifesto’s “hyperplane LSH” with learnable anchor vectors, 1, §VIII-H, is the same form; the pair comparison is its special case with one $+1$ and one -1).
- **The output projection compresses the tables.** Tables live in a narrow d_{out} (48 in every experiment below); **decompress** expands the summed read back to model width, so the cells stay small.

Given a learned input projection, the single-anchor mode alone would have sufficed in principle: a pair comparison $z[a] - z[b] > 0$ is itself a hyperplane on x , so anchor pairs could have been dropped and left to **compress** to realise. The library keeps both modes deliberately. The pair comparison is the Spiking Manifesto’s mechanism as stated — the relative order of two coordinates — and it is the cheap special case that the whole design is built to exploit: an address bit costs two coordinate reads and one subtraction, with the anchors stored as integer indices and no weights of its own (the kernels do exactly this), whereas a general per-table hyperplane would cost a d_{in} -length dot

¹The two head counts may differ only when one of them is 1; the library rejects every other combination. With $h_{\text{in}} = 1$ the input projection produces one d_{in} -wide slice that all G heads read (fan-out); with $h_{\text{out}} = 1$ the heads’ outputs are summed into one d_{out} -wide slice that the output projection consumes (fan-in).

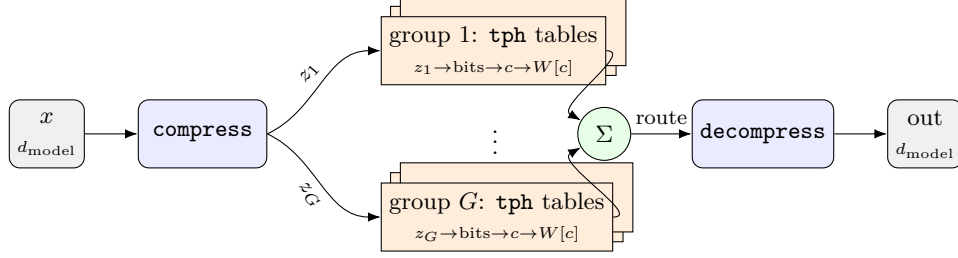


Figure 2: The ProjectionLUT data path. The input projection **compress** ($d_{\text{model}} \rightarrow h_{\text{in}} d_{\text{in}}$) splits x into per-group spaces z_g ; in each group the probes hash z_g to a cell per table and the **tph** reads are summed; the group outputs are routed and the output projection **decompress** ($h_{\text{out}} d_{\text{out}} \rightarrow d_{\text{model}}$) expands the result. The input and output widths are independent; both are d_{model} in every configuration here. The box in the middle is the cartridge; the wrapper treats it as opaque.

product and d_{in} weights per bit — at the sweep geometry, some 4,000 subtractions per token and layer against about 200,000 multiply-adds. Single anchors were added later for flexibility, as the even cheaper probe, and sit at the other end of the same trade.

Either side may be switched off (Identity) when the widths already match. **decompress** is initialised to zero and **compress** to $\mathcal{N}(0, 0.02)$, so a freshly built layer contributes nothing and the tables start learning only once **decompress** has moved off zero — the usual zero-init of a residual branch.

3 Cartridges: one contract, three generations

Everything above is fixed: the geometry, the addressing, the sum over tables, the routing, the wrapper. A *cartridge* is the swappable read *mathematics* inside that frame. The contract is a shape: $[B, h_{\text{in}}, d_{\text{in}}] \rightarrow [B, h_{\text{out}}, d_{\text{out}}]$ (**MultiHeadLUT**), and the wrapper never reaches inside it. Every shipped cartridge inherits one base, **ManifestoLUT**, which owns the tables $W \in \mathbb{R}^{G \times \text{tph} \times K \times d_{\text{out}}}$, the anchors (drawn once from a seed and never trained), the addressing (c, u, c') , the sum over **tph** and the routing; what a cartridge adds is how it turns the addressed cell, its neighbour and the margins into a per-table output — and which function training differentiates.

Three generations of one idea. The cartridges form a progression. Gen-1 *Manifesto* is the Spiking Manifesto’s model as published: a hard read and a fixed uncertainty function for the gradient. Gen-2 *SoftSign* replaces the fixed uncertainty by a blend weight with two learned temperatures. Gen-3 *Confidence* replaces the blend by a learned *score* that weights each table by how confident its address is — and has an int8 twin for deployment. Table 1 is the map.

Pure and fused twins. Each generation has a *pure* cartridge — readable PyTorch, the numerical oracle — and a *fused* twin with the same mathematics that dispatches each forward to the fastest available path: a single `F.embedding_bag` over the addressed cells, or co-located CUDA kernels (§5). Fused twins are tested against the pure ones; the sweep in §6 runs the fused twins where they exist.

cartridge	gen	forward value (per table)	gradient / surrogate	learned scalars
ManifestoHardLUT / Fused...	1	$W[c]$	input through the blend $(1 - U)W[c] + U W[c']$, $U = 0.5/(1 + u^*)$; weights: row c only	—
ManifestoSoftLUT / Fused...	1	$(1 - U)W[c] + U W[c']$	the value itself; weights: both rows	—
SoftSignHardLUT / Fused...	2	$W[c]$	input and temperatures through $(1 - w)W[c] + w W[c']$; weights: row c	$T_{\text{soft}}, T_{\text{sel}}$
SoftSignSmoothLUT / Fused...	2	$(1 - w)W[c] + w W[c']$	the value itself; weights: both rows	$T_{\text{soft}}, T_{\text{sel}}$
ConfidenceLUT ($n=1$)	3	$s W[c]$	the value itself; input only through s	β, γ
ConfidenceLUT ($n=2$)	3	$s[(1 - v)W[c] + v W[c']]$	the value itself; input through s and v	β, γ, τ
QuantisedConfidenceLUT	3	as above with int8 rows and power-of-two weights	straight-through to the float masters	β, γ, τ

Table 1: The cartridge matrix. c : addressed cell; c' : least-certain-bit neighbour; u^* : the deciding margin; “Fused...” marks a fused twin with identical mathematics. “Weights: row c only” means the surrogate shapes the input gradient but the table update stays on the addressed row.

4 The cartridges

Each subsection follows one template: the value, the gradient, what is learned, when to use it, and the three lines that build it. Throughout, `spec` is a `LUTSpec` — the layer’s geometry in one frozen dataclass: heads, tables per head, probes per table, widths and probe kind (Appendix A) — and `ffn = ProjectionMHL(cart, d_model)` wraps any cartridge.

4.1 Manifesto: ManifestoHardLUT, ManifestoSoftLUT

The Spiking Manifesto’s model and learning rule, with its uncertainty function.

Value. The hard read $y = \sum_t W_t[c_t]$ — one row per table, added, no multiply (manifesto Eqs. 1–3). **Gradient.** The addresses are integers, so the paper differentiates a surrogate instead (its Eq. 4): the two-cell blend

$$\tilde{y} = \sum_t [(1 - U_t) W_t[c_t] + U_t W_t[c'_t]], \quad U_t = U(u_t^*) = \frac{0.5}{1 + |u_t^*|},$$

whose derivative lands on the two anchors of the deciding probe alone (Eq. 5): $\partial \tilde{y} / \partial z[a] = -\partial \tilde{y} / \partial z[b] = U'(u^*) (W[c'] - W[c])$. U is the paper’s own choice; $U(0) = \frac{1}{2}$, $U \rightarrow 0$ for a confident bit. **Learned:** the tables only.

Hard vs. soft, and provenance. **ManifestoHardLUT** is Izhikevich’s rule verbatim: hard value at train and eval, the addressed row updated, the input gradient through the minimal-pair surrogate Eq. 4 with $U = 0.5/(1 + |u|)$ (1, §III-A and Fig. 10). **ManifestoSoftLUT** goes one step beyond the paper: it uses Eq. 4 itself as the forward value at train *and* eval, so both the addressed cell and its neighbour receive weight gradient. The manifesto states that U is used “only in the training mode to calculate gradients” and that inference uses Eq. 3 without U (§III-A); its appendix explores other surrogates of the same family — all pairs, or dropping the flipped row, $y = \sum S_x(1 - U)$

(§VIII-K, Eqs. 17–18) — again only as learning rules. In the sweep the soft read is the better of the two by 0.012 bpb (Table 2).

Fused twins. `FusedManifestoHardLUT` keeps the exact straight-through backward in a hand-written scatter: the weight gradient goes to row c by `index_add`, the input gradient is $U'(u^*)$ times the alignment of the output gradient with $W[c] - W[c']$, scattered to the deciding pair with opposite signs. `FusedManifestoSoftLUT` reads with `embedding_bag` and per-sample weights $[1-U, U]$. Both can route training to the native `lprojection` kernels. **When:** the reference points; the hard read is the cheapest possible inference.

```
from spiky.lutorch_ex import LUTSpec, FusedManifestoHardLUT, FusedManifestoSoftLUT, ProjectionMHL
spec = LUTSpec(h_in=8, h_out=8, tph=64, nap=8, d_in=48, d_out=48)
ffn = ProjectionMHL(FusedManifestoSoftLUT(spec, seed=1), d_model=384)
```

4.2 SoftSign: `SoftSignHardLUT`, `SoftSignSmoothLUT`

Gen-2 keeps Gen-1’s structure and replaces the fixed U by a blend weight with two learned temperatures per layer (the two-cell form of the soft-sign mechanism in 2, §2):

$$\rho = \frac{|u^*|}{T_{\text{soft}} + |u^*|} \in (0, 1), \quad w = \sigma(-2\rho/T_{\text{sel}}) \in (0, \tfrac{1}{2}] \quad (\text{weight on the neighbour } c').$$

Value / gradient. `SoftSignHardLUT`: hard read, surrogate $(1-w)W[c] + wW[c']$ for the input and the temperatures, weights updated on row c . `SoftSignSmoothLUT`: the blend is the value at train and eval, both rows learn. Only the two cells $\{c, c'\}$ enter; a softmax over all K cells and several-neighbour forms are deliberately out of scope. **Learned:** tables, $\log T_{\text{soft}}$, $\log T_{\text{sel}}$ (init 0.5; freezable). **Fused twins.** `FusedSoftSignHardLUT` runs its training step on the native kernels plus one small surrogate-gradient kernel (a clear win: $1.6\times$ faster step, $2.7\times$ less memory than the pure cartridge at the training batch); `FusedSoftSignSmoothLUT`’s best path is the `embedding_bag` one, so that is what its default dispatch picks. **When:** when a learned sharpness matters and the score of Gen-3 is not wanted.

```
ffn = ProjectionMHL(FusedSoftSignSmoothLUT(spec, seed=1, soft_score_temp=0.5, select_temp=0.5),
    d_model=384)
```

4.3 Confidence: `ConfidenceLUT`

Gen-3 does not blend by uncertainty — it *weights* each table by how confident its address is, with the learned-margin score of LookupFFN [5] (derived in 2, §3, §6). With margins $m_j = |u_j|$,

$$s = \left(\sum_j m_j \right) \exp\left(\gamma \sum_j \log \sigma(\beta m_j) \right), \quad \beta = e^{\log \beta}, \quad \gamma = e^{\log \gamma} \text{ learned per layer (init 2, 1).}$$

Value. $n=1$: $y = \sum_t s_t W_t[c_t]$. $n=2$: the scored two-cell blend $y = \sum_t s_t [(1-v_t)W_t[c_t] + v_t W_t[c'_t]]$ with $v = \sigma(-2|u^*|/\tau) \in (0, \tfrac{1}{2}]$ the weight on the *neighbour*², τ learned per layer (init 0.5). **Gradient.** The value itself. The address is a detached integer, so the input gradient reaches z *only* through s (all `nap` margins) and, for $n=2$, through v (the deciding margin): a table learns how confident to be, never that a different row would have been better. **Learned:** tables, $\log \beta$, $\log \gamma$,

²The code and the ablation table write v on the neighbour, $v = \sigma(-2|u^*|/\tau)$. The quantisation note [4] writes the same blend the other way, $v = \sigma(2u^*/\tau)$ on the addressed cell; the values agree since $\sigma(x) + \sigma(-x) = 1$.

and $\log \tau$ for $n=2$. **Implementation.** One `embedding_bag` with the scores as per-sample weights; the whole training forward is `torch.compile` on CUDA, which folds the score’s chain of small kernels and is what makes this cartridge’s training step the cheapest of the three (§6.1). **When:** the best accuracy of the three generations (Table 2); fp32/fp64 only.

```
cart = ConfidenceLUT(spec, seed=1, read_top_n=2, beta_init=2.0, gamma_init=1.0,
                    read_tau_init=0.5, read_tau_learnable=True)
ffn = ProjectionMHL(cart, d_model=384)
```

4.4 Quantised confidence: QuantisedConfidenceLUT

The same cartridge trained so that the deployed read needs only int8 rows, additions and bit shifts (`quant_mode="p2_int8"`; the derivation is 4, here only the three facts).

1. **Rows are int8 with a power-of-two scale** shared by all tables of a group and one output channel: $e_{g,k} = \lceil \log_2 \max |W| \rceil - 7$, $\hat{W} = \text{clamp}(\text{round}(W/2^e), -128, 127)$. Training uses $2^e \hat{W}$ in value with a straight-through gradient to the float master W .
2. **The two blend weights are rounded to powers of two.** With $r = (1 - v)/v = e^{-2|u^*|/\tau}$ the read factorises as $s(1 - v)(W[c] + r W[c'])$; $q = \text{round}(2|u^*|/(\tau \ln 2))$ rounds r to 2^{-q} and $k' = \text{round}(\log_2 s - c_q)$ rounds the rest, so a table contributes $2^{k'} W[c] + 2^{k'-q} W[c']$. Again straight-through: the forward uses the rounded weights, the gradient is that of $s(1 - v)$ and $s v$, so β, γ, τ keep learning.
3. **The integer read.** In units of 2^{-6} every shift is a left shift, the int32 sum fits, and the per- (g, k) scale 2^{e-6} is the only float left — and it is applied *after* the table sum.

Exact constants (window, cutoffs, drop rule) are in Table 4 of the appendix. The train-mode read equals the deployed read bit for bit; `forward_int` performs the int32 shift-add for cross-checking. **Cost:** +0.0012 bpb against the float cartridge in the 48k sweep, under the +0.003 upper bound of the note; tables $4\times$ smaller. **When:** whenever the model will be deployed. $n=2$ is the reference integer form; $n=1$ is a library extension.

```
cart = QuantisedConfidenceLUT(spec, seed=1, quant_mode="p2_int8", read_top_n=2,
                             beta_init=2.0, gamma_init=1.0, read_tau_init=0.5)
```

4.5 Deployment: the .lxq format

```
from spiky.lutorch_ex import export_deployment, load_deployment
export_deployment(model, "m.lxq") # int8 rows, scale folded, fp32 dropped
dep = load_deployment("m.lxq", make_skeleton, device="cuda").eval() # the fp32 table is never
    allocated
```

`export_deployment` walks the model and, for every `ProjectionMHL` whose cartridge emits a compact payload (`to_deployment()`, a protocol the base answers with “dense, nothing to do”), writes the int8 rows, the anchors and the three scalars, and **folds the scale 2^{e-6} into a copy of that layer’s decompress weight**. The fold is exact — a power of two only touches the exponent field of a float — and legal because nothing sits between the table sum and `decompress` and one exponent is shared across a group’s tables, so it factors out of the sum (the same idea as the `SupportsDecompressBake` hook of the wrapper, applied once and statically at export; the runtime hook is implemented by no shipped cartridge). The loader builds the architecture on the `meta`

device, rebuilds each compacted layer from a format tag through a small registry (so `deploy.py` names no cartridge class), and loads the rest of the state dict. The deployed cartridge holds only the int8 `packed` buffer and returns the raw int32 accumulator; the file is a safetensors container with a JSON header. Per layer at the sweep geometry: 24 MiB fp32 $\rightarrow$ 6 MiB int8. Scope today: $n=2$ and $h_{\text{out}} = G$.

5 Shared machinery

Regularisers (both default off, inherited by every cartridge). *Cell total variation:* view a table’s $K = 2^{\text{nap}}$ cells as the vertices of an `nap`-cube and penalise the mean squared difference over all Hamming-1 pairs, $\mathcal{R}_{\text{TV}} = \langle \|W_c - W_{c'}\|_2^2 \rangle_{(c,c')}$ (the mean over the $n_{\text{tables}} \cdot \text{nap} \cdot 2^{\text{nap}-1}$ pairs) — a locality prior: a one-bit address flip should change the read little. `cell_tv_penalty(model)` averages it over the model’s cartridges; add $\lambda \mathcal{R}_{\text{TV}}$ to the loss. *Table dropout (table_dropout_rate):* inverted dropout at whole-table granularity, one Bernoulli per (sample, group, table), training only; a dropped table contributes nothing to the value or the gradient.

Precision. The pure cartridges are fp32/fp64 and raise on bf16/fp16. The fused Manifesto and SoftSign twins accept bf16/fp16: addressing runs in fp32 (so a lossy margin cannot flip a bit), reads and reductions accumulate in fp32, the output is cast once. Gen-3 is fp32/fp64.

Compilation and kernels. Eval forwards are `torch.compile` on CUDA; training forwards are compiled only for Gen-3, where it pays. The fused twins resolve one native extension, `lutorch_ex_lprojection` (the lookup and anchor-pair kernels under `cartridges/csrc/`, JIT-built with `torch.utils.cpp_extension`), plus three small standalone kernels (int8 scalars, single-anchor input gradient, soft-sign surrogate). Each is lazy, cached across processes, and falls back to pure torch on CPU or without `nvcc/ninja`, with identical results. A `backend=` argument forces `pure`, `tier1` or `native` on any fused twin; `bench.py` times the forward, training forward and backward of any configuration.

6 Experiments: the seven-configuration sweep

Setup. A six-block GPT ($d_{\text{model}} = 384$, 6 attention heads, 68.2M parameters) with the feed-forward layer of every block replaced by a `ProjectionMHL` around the cartridge under test. All seven runs share the champion topology — $h_{\text{in}} = h_{\text{out}} = 8$, `tph`= 64, `nap`= 8, $d_{\text{in}} = d_{\text{out}} = 48$, anchor pairs — seed 1, the default init (per-head Uniform $[\pm 10^{-3}]$ tables, `decompress` zeroed), cell-TV $\lambda = 10$, table dropout 0.2, AdamW (3×10^{-4} , weight decay 0.1, none on the tables), 48,000 steps, cosine schedule with 10% warm-up to a $0.1 \times \text{floor}$, sequence length 512, 24,576 tokens per step (device batch 12, accumulation 4), evaluation every 500 steps; the backbone runs in bf16, the LUT layers in fp32. Data and tokenisation are the *NanoChat* setup: the ClimbMix text corpus as prepared by nanochat’s data script, streamed and tokenised on the fly by nanochat’s RustBPE tokenizer (32,768-entry vocabulary, trained by nanochat on the same corpus) into BOS-aligned 512-token rows; `val_bpb` is the validation cross-entropy in bits divided by the number of *bytes* of the decoded text (nanochat’s bits-per-byte evaluation on a fixed held-out set), so it is comparable across tokenisers. Per-run configs, metric histories and summaries are under `experiments/lutorch_ex/`.

³Best over the 96 evaluations. For the three runs reconstructed from the tracking server (`FusedManifestoHard/Soft`, `FusedSoftSignHard`) the final values are 0.0004–0.0007 higher than the best; for the

feed-forward layer	read	params	best val_bpb ³	Δ vs. dense
dense FFN, $384 \rightarrow 1536 \rightarrow 384$, GELU	—	35.8M	1.11509	—
ConfidenceLUT $n=2$	scored two-cell blend, float	68.2M	1.10606	−0.0090
QuantisedConfidenceLUT $n=2$	the same, int8 + power-of-two	68.2M	1.10724	−0.0079
ConfidenceLUT $n=1$	scored single cell	68.2M	1.11737	+0.0023
FusedSoftSignSmoothLUT	learned-temperature blend	68.2M	1.12941	+0.0143
FusedManifestoSoftLUT	U -blend	68.2M	1.13949	+0.0244
FusedManifestoHardLUT	hard read	68.2M	1.15152	+0.0364
FusedSoftSignHardLUT	hard read	68.2M	1.15647	+0.0414

Table 2: The 48k sweep, one seed, against the dense feed-forward baseline: the same backbone (6 blocks, $d_{\text{model}} = 384$, 6 attention heads), data, tokenizer, schedule, 24,576-token batches and seed, with the dense $384 \rightarrow 1536 \rightarrow 384$ GELU layer in the feed-forward slot (run `exp_n_0177_vanilla48k_corrected` on branch `research/ffn_replacement_fix`). Run identifiers, hosts, wall-clock and peak memory of the seven LUT runs are in Table 5 and in `experiments/lutorch_ex/README.md`.

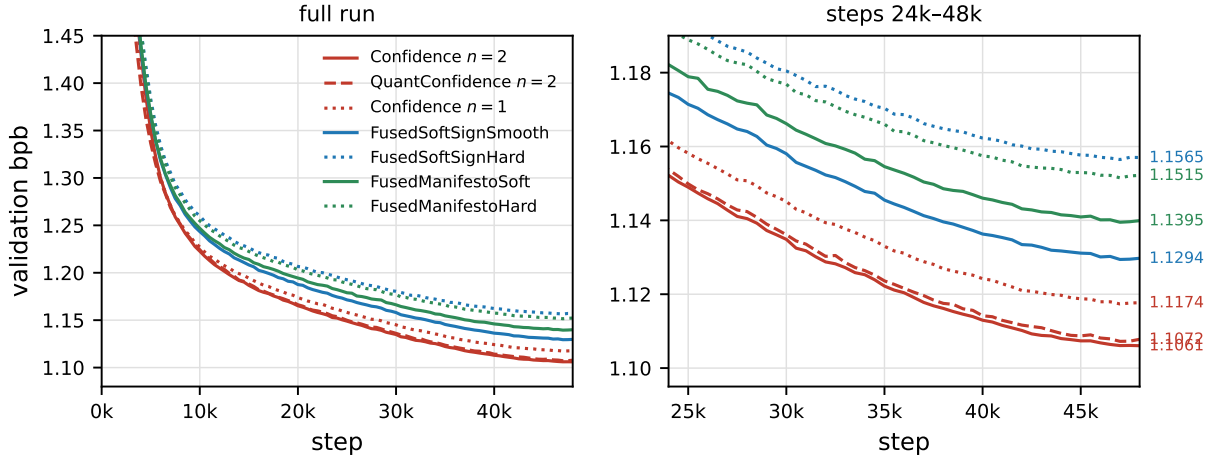


Figure 3: Validation bpb of the seven runs (left: full run; right: the last half, with each run’s best value). Colour is the generation (red Confidence, blue SoftSign, green Manifesto); dotted lines are hard reads. Generated by `make_fig_curves.py` from the committed `metrics.csv` files.

Reading the results. Four things are visible in Table 2 and Figure 3. (i) A *soft* read beats its hard counterpart in every generation: −0.012 (Manifesto), −0.027 (SoftSign), −0.011 (Confidence $n=2$ vs. $n=1$). (ii) A *learned confidence score* beats a blend by uncertainty: the Confidence line leads the SoftSign line by 0.023 and the Manifesto line by 0.033. The two effects stack, and the ordering is fixed from about step 8k onwards. (iii) *Quantisation is nearly free*: int8 rows and power-of-two weights cost +0.0012 bpb, well under the +0.003 bound of the quantisation note, for tables one quarter the size. (iv) Against the dense feed-forward layer, the two scored two-cell cartridges *win* (−0.009 and −0.008 bpb), the single-cell scored read is within 0.002, and the blend-only generations trail by 0.014–0.041; the LUT models carry $1.9\times$ the parameters of the dense one, but a LUT layer touches only two 48-wide rows per table per token.

other four best and final coincide. The baseline’s final value is 1.11542.

6.1 Benchmarks: CPU, RTX 5090, H100

Table 3 times the layer alone, at the sweep geometry, for the seven optimised configurations the sweep used: the four fused twins and the three Gen-3 configurations (whose `embedding_bag` + compiled forward is already the optimised path; the pure reference cartridges are not benchmarked). The CPU and RTX 5090 columns were measured for this report with the library’s own harness (`bench.py`, driven by `bench_report.py` next to this report); the H100 column is carried over from the benchmark recorded in the pull request that landed the Gen-3 cartridges⁴, and is not re-measured here — that machine is gone.

cartridge	CPU (measured)	RTX 5090 (measured)		H100 (recorded, PR #138)			
	step ms, $B=1024$	eval ms	step ms	peak GB	eval ms	step ms	peak GB
FusedManifestoHardLUT	62.9	1.04	20.2	3.8	1.61	21.3	4.0
FusedManifestoSoftLUT	92.3	3.50	22.5	2.3	3.84	23.6	2.5
FusedSoftSignHardLUT	68.8	1.03	24.6	4.2	1.61	23.8	4.2
FusedSoftSignSmoothLUT	60.8	2.13	28.8	2.9	4.10	28.4	3.0
ConfidenceLUT $n=1$	35.8	1.51	11.5	1.1	2.40	12.9	1.2
ConfidenceLUT $n=2$	66.7	4.59	21.5	1.9	4.17	24.7	2.1
QuantisedConfidenceLUT $n=2$	83.4	4.12	23.9	1.8	4.23	24.5	2.0

Table 3: Layer-only timings at $h_{\text{in}} = h_{\text{out}} = 8$, $\text{tph} = 64$, $\text{nap} = 8$, $d_{\text{in}} = d_{\text{out}} = 48$, fp32, random input. *eval* = no-grad forward; *step* = training forward + backward. GPU columns at $B = 24,576$ (the sweep’s training batch). **Measured here:** one workstation, torch 2.9.1 (cu130), CUDA 13.3 toolkit for the JIT kernels, driver 595; RTX 5090 (32 GB, native kernels active): medians of 10 repeats after 3 warm-ups; CPU (24-thread desktop part, the fused twins’ `embedding_bag` path, $B = 1,024$): medians of 15 repeats after 5 warm-ups, with the fastest repeat within 15% of the median (the committed `bench_cpu.json` and `bench_cuda.json` hold every cell). **Carried over:** the H100 column, verbatim from the recorded benchmark cited in the text, not re-measured.

Three readings. On either GPU the training step of every blend or hard cartridge is 20–29 ms at the training batch and is dominated by the backward (17–19 ms), the scatter into the 6.3M-entry table; **ConfidenceLUT $n=1$** is both the cheapest step (11.5 ms) and the smallest footprint (1.1 GB), because its compiled forward fuses the score’s chain of small kernels and reads one cell. Reading two cells ($n=2$, the soft forms) roughly doubles the eval forward (1.0 $\rightarrow$ 3.5 ms for Manifesto, 1.5 $\rightarrow$ 4.6 ms for Confidence). The quantised cartridge’s eval forward is not faster than the float one — its training-time read still fetches float rows for the straight-through estimator; the integer read belongs to the deployed cartridge (§4.5). The two GPUs agree to within 15% on every row: these layers are bound by memory traffic, not arithmetic. On the CPU, at a batch $24\times$ smaller, the same layers take 36–92 ms per step — roughly two orders of magnitude slower per row — with the same ordering: the one-cell scored read is the cheapest, the two-cell blends cost $1.7\text{--}2.6\times$ more.

⁴PR #138, the “full cross-generation benchmark” comment, commit 996046ce: H100 80 GB, fp32, each cartridge in its own process, 25 warm-up + 60 timed repetitions, GPU clock held at 1980 MHz. PR #137 reports the four fused Gen-1/2 twins on the same card with the same harness and agrees within 2%. Those recorded numbers are process-peak allocations; the RTX 5090 peak here is the allocation added by one training step. The deployed int8 cartridge (§4.5) has no training row; its recorded eval forward (PR #139, H100, $B = 4,096$) was 2.6 ms against 2.8 ms for the float twin.

7 Status and what is next

- **One seed.** Every row of Table 2, the dense baseline included, is a single seed; differences below about 0.003 bpb are inside seed noise at this scale.
- **Single-anchor mode** (§1) is implemented and tested for every cartridge but none of the seven runs uses it.
- **Deployment** covers $n=2$ and $h_{\text{out}} = G$ only; fan-in would need a pre-route scale. int4 is defined by the quantisation note but untested in this library.
- **Benchmarks** (§6.1) are single-process microbenchmarks of the layer alone; an end-to-end training-throughput and inference-latency measurement of the whole model is still to do.

References

- [1] E. M. Izhikevich. “Spiking Manifesto.” arXiv:2512.11843, v2, December 2025. Vendored as `sources/spiking_manifesto_izhikevich_arxiv2512.11843.pdf`.
- [2] DeepSpike Inc. “The Mathematics of Our LUT Mechanisms.” Internal note, 2026. Vendored as `sources/lut_mechanisms.tex`.
- [3] “LUT core-module ablation: three generations of lookup math.” Internal note, 2026-09-14. Vendored as `sources/lut_ablation_table_v4.tex`.
- [4] “LUT read and quantisation (simplified).” Internal note, 2026-10-04. Vendored as `sources/quantisation_simple_norowops.md`.
- [5] Z. Zeng, M. Davies, P. Pulijala, K. Sankaralingam, V. Singh. “LookupFFN: Making Transformers Compute-lite for CPU Inference.” ICML 2024.

A Reference

Geometry and constructor arguments.

LUTSpec	<code>h_in, h_out, tph, nap, d_in, d_out, anchor_mode</code> ; allowed head patterns $h_{\text{in}} = h_{\text{out}}$, $h_{\text{in}} = 1$ (fan-out), $h_{\text{out}} = 1$ (fan-in)
ManifestoLUT (all)	<code>seed</code> (anchors and tables), <code>weight_init_std</code> (default 10^{-3} : per-head Uniform[$\pm$ std]), <code>cmp_eps</code> , <code>table_dropout_rate</code> , <code>device</code>
SoftSign	<code>soft_score_temp</code> , <code>select_temp</code> (init 0.5), <code>learnable_temps</code>
Confidence	<code>read_top_n</code> $\in \{1, 2\}$, <code>beta_init</code> , <code>gamma_init</code> , <code>read_tau_init</code> , <code>read_tau_learnable</code> , <code>learnable_score</code>
Quantised	<code>quant_mode="p2_int8"</code> , <code>quant_overrides</code>
Fused twins	<code>backend</code> $\in \{\text{auto}, \text{pure/pure_eval}, \text{tier1}, \text{native}\}$
ProjectionMHL	<code>input_dim</code> , <code>output_dim</code> (may differ; <code>d_model</code> sets both), <code>compress/decompress</code> (bool), <code>bias</code> , <code>device</code>

Quantisation constants (p2_int8).

bits / offset	$b = 8, o = 0$ (int4: $b = 4, o = -1$)
blend exponent q	$\text{clamp}(\lfloor 2 u^* /(\tau \ln 2) + \frac{1}{2} \rfloor, 0, J=64)$; $c_q = \log_2(1 + 2^{-q})$ for $q < C=8$, else 0; second cell dropped when $q > Q=3$
weight exponent k'	$\text{round}(\log_2 s - c_q)$ in the window $[-3, 4]$: below $\rightarrow$ table skipped (gradient kept), above $\rightarrow$ clamped
fixed point	accumulator in units 2^{-6} (FIXED_POINT_SHIFT= 6), so all shifts are left shifts

Table 4: The constants of the power-of-two read, from [4].

The seven runs.

run folder (suffix after <code>lutorch_ex_abl47_</code>)	cartridge	hours	peak GB
<code>conf_1004_1148</code>	<code>ConfidenceLUT $n=2$</code>	3.8	7.7
<code>quant_1004_1148</code>	<code>QuantisedConfidenceLUT $n=2$</code>	4.8	7.7
<code>conf_n1_vm_1004_1806</code>	<code>ConfidenceLUT $n=1$</code>	2.9	7.7
<code>fss_smooth_vm_1004_1806</code>	<code>FusedSoftSignSmoothLUT</code>	4.2	7.7
<code>fms_wk_1004_1510</code>	<code>FusedManifestoSoftLUT</code>	3.7	7.7
<code>fmh_wk_1004_1510</code>	<code>FusedManifestoHardLUT</code>	4.7	8.5
<code>fss_hard_wk_1004_1510</code>	<code>FusedSoftSignHardLUT</code>	5.2	9.4

Table 5: Wall-clock and peak device memory per run, from each run’s `summary.json` under `experiments/lutorch_ex/`; the `vm` and `wk` runs trained on two different cloud hosts.

Environment and layout. `LUTORCH_EX_NO_COMPILE=1` disables `torch.compile`, `LUTORCH_EX_NO_CUDA_EXT=1` skips the JIT kernels, `TORCH_EXTENSIONS_DIR` relocates their build cache. Under `src/spiky/lutorch_ex/`: the geometry, contract, addressing, anchors, wrapper, deployment and benchmark modules at the top level; one module per cartridge under `cartridges/` (shared base `manifesto_base.py`, quantisation mathematics `_pow2.py`, kernel sources `csrc/`); 17 test files under `tests/`.